



Benutzerverwaltung über SCIM:

So geht es richtig



Agenda

- 01 | Was ist SCIM?
- 02 | Herausforderungen
- 03 | Implementierungsansatz
- 04 | Überraschung
- 05 | Fazit

Über mich



- › Softwarearchitekt (Azure, SAS Viya, & SAS 9)
- › seit über 15 Jahren Projekte im SAS-Umfeld
- › aktueller Schwerpunkt: Agentic AI

01 | Was ist SCIM?

Kontext: SAS Viya benötigt Benutzerinformationen

Benutzerverzeichnis enthält Infos zu: Name, Emailadresse, Gruppen, usw.

Option 1:
LDAP (bzw. AD)



Option 2:
SCIM

SCIM: System for Cross-domain Identity Management

Anwendungsfall:

- › Offener Standard zur **automatisierten Verwaltung von Benutzeridentitäten**
 - › Ziel: **Benutzer, Gruppen und Rollen** systemübergreifend synchronisieren
 - › Entwickelt für **Cloud- und SaaS-Umgebungen**
 - › Standardisiert durch die IETF (Internet Engineering Task Force)
- SCIM ermöglicht konsistente Nutzerdaten in verteilten Systemen/Anwendungen

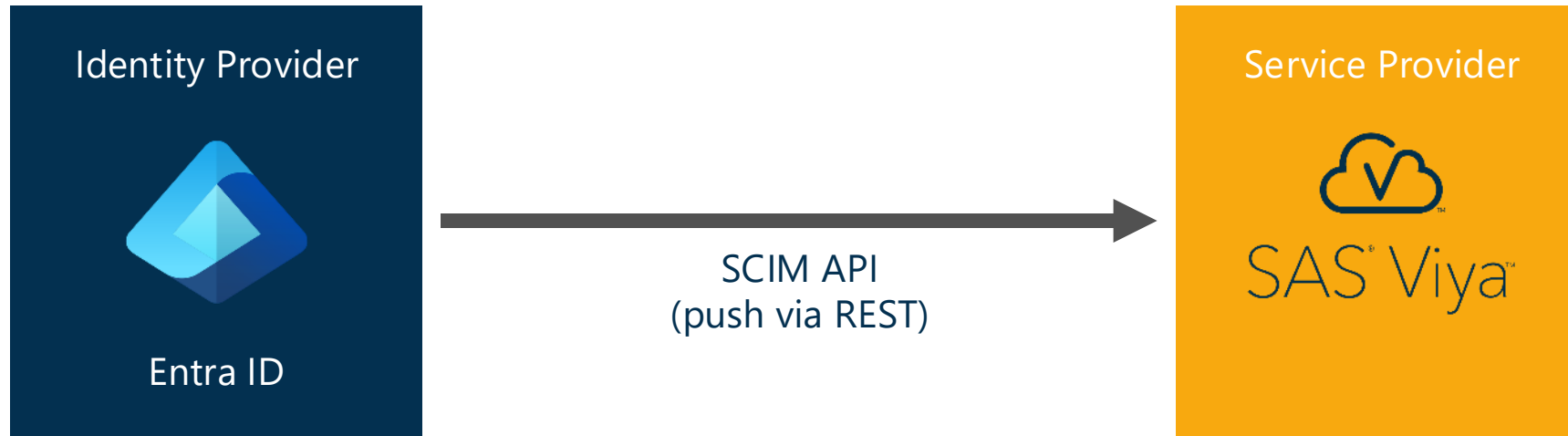
SCIM basiert auf **offenen Standards**:

- › REST API (HTTP/S)
- › JSON

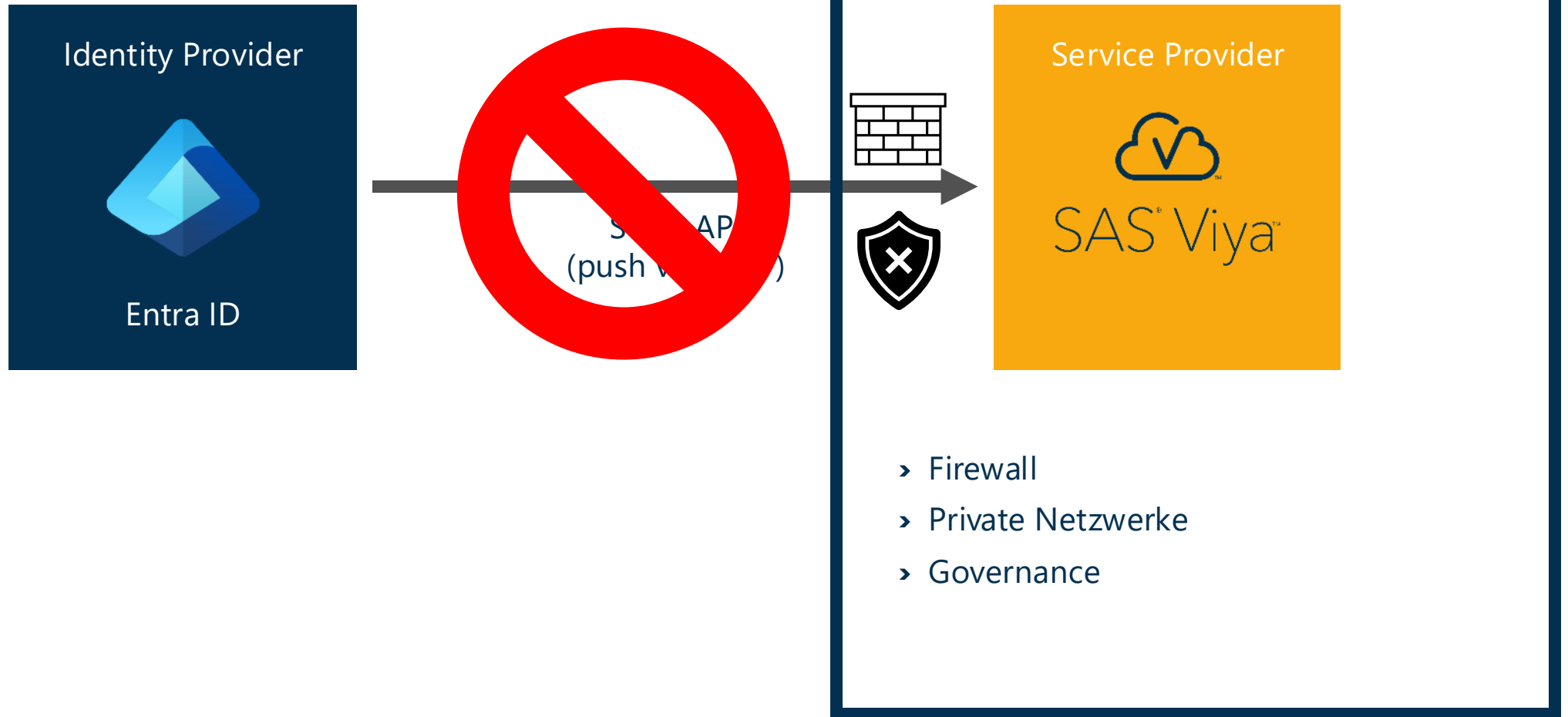


02 | Herausforderungen

Typische SCIM Architektur

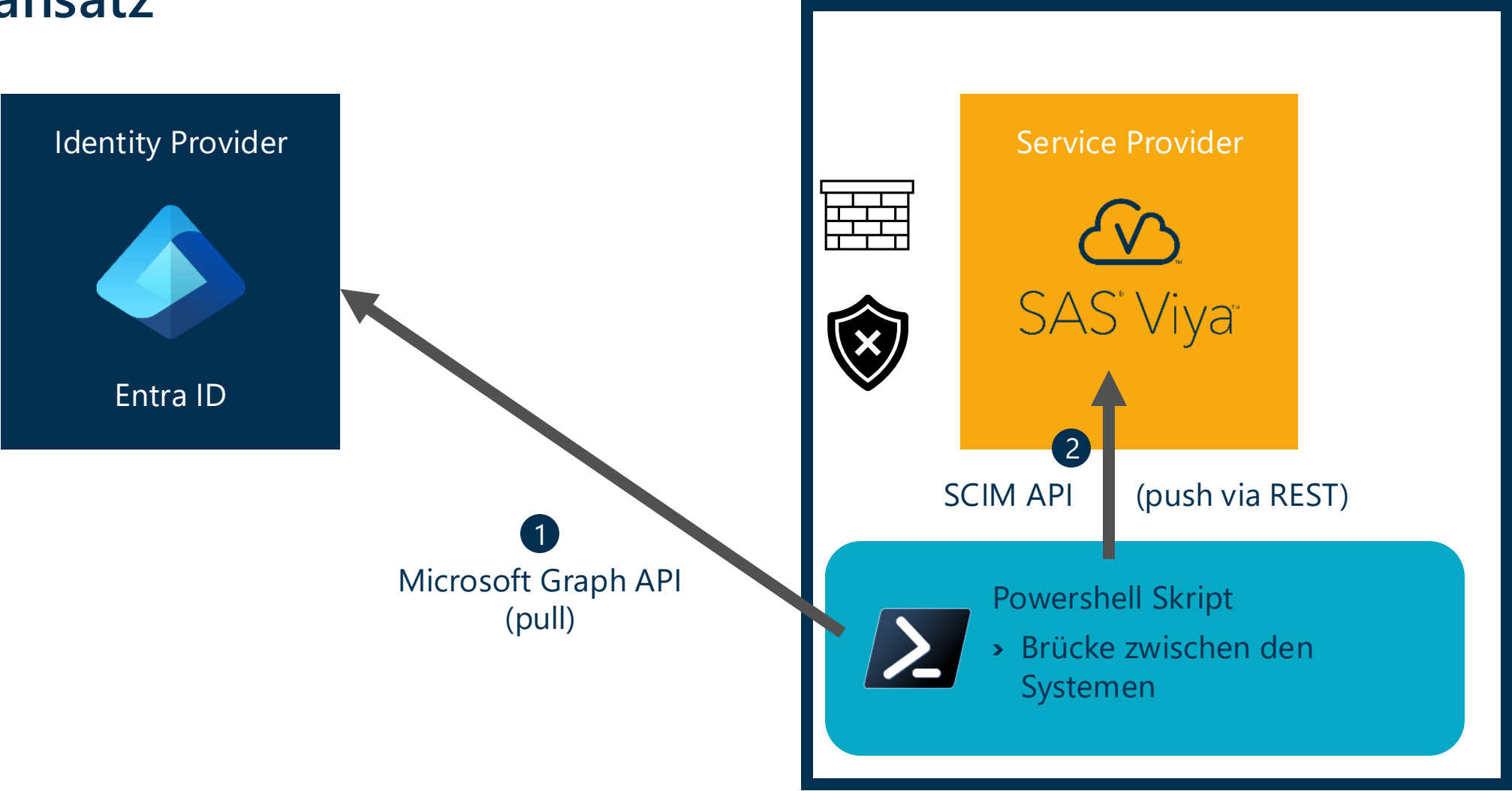


Herausforderung



03 | Implementierungsansatz

Lösungsansatz



Der Ablauf



GitHub Workflow (tägliche Ausführung)

Zielzustand aus Entra ID
abfragen

CSV Export für Audit:
Benutzer, Gruppen,
Gruppenzuordnungen

Abgleich zwischen CSVs und
Viya: create, update, delete

Auszug aus Audit Log

applying state steps:

importing state files:

✓ importing state files was successfull

get secrets from key vault:

✓ getting key vault secrets was successfull

Viya login:

✓ Viya login successfull

diff users:

users to create: 0
users to update: 39
users to delete: 0

diff groups:

groups to create: 0
groups to delete: 0

Viya syncing group memberships:

✓ group members for group PROJECT_A synchronized
✓ group members for group PROJECT_B synchronized
✓ group members for group PROJECT_C synchronized
✓ syncing memberships was successfull

Summary:

✓ all steps completed successfully

Codebeispiele

Benutzer aus mehreren Entra Gruppen per Graph API auslesen

```
$groups_with_users_to_sync = @"(GROUP1", "GROUP2")

Write-Output "## Fetching users from Graph API:"
try {
    # login to graph api using service principal
    Connect-MgGraph -TenantId $tenantID -ClientSecretCredential $ClientSecretCredential
    ....
    $users = @()
    foreach ($groupName in $groups_with_users_to_sync) {
        # resolve group name to group id
        $groupId = Get-MgGroup -Filter "displayName eq '$groupName'" | Select-Object -ExpandProperty Id

        # query group members
        $users += Get-MgGroupMember -GroupId $groupId -All |
        Select-Object Id, `
        @{label="UserPrincipalName"; expression={ $_.AdditionalProperties.userPrincipalName }}, `
        @{label="DisplayName"; expression={ $_.AdditionalProperties.displayName }}, `
        @{label="Mail"; expression={ $_.AdditionalProperties.mail }}
    }

    # keep only unique users
    $users = $users | Sort-Object -Property UserPrincipalName -Unique

    Write-Output "✅ fetching users was successful"
}
catch {
    Write-Output "❌ fetching users failed"
    Exit 1
}
```

Codebeispiele

SCIM/Viya: Benutzer auslesen

```
try {  
    $headers = @{  
        'Authorization' = "Bearer $access_token"  
    }  
  
    $viya_users_response = Invoke-RestMethod `~  
        -Uri "$viya_url/identities/scim/v2/Users?count=10000" `~  
        -Headers $headers `~  
        -Method Get `~  
        -SkipCertificateCheck `~  
        -StatusCodeVariable "statuscode" `~  
    `~  
    if ($statuscode -in (200, 201, 204)) {  
        $viya_users = $viya_users_response.Resources | Select-Object Id, externalId, UserName, DisplayName, active  
        foreach ($viya_user in $viya_users) {  
            Write-Output "👤 $($viya_user.UserName) [ $($viya_user.DisplayName) ] ( $($viya_user.Id) )"  
        }  
    }  
    else {  
        throw "error"  
    }  
    `~  
    Write-Output "✅ listing users was successfull"  
}   
catch {  
    Write-Output "❌ listing users failed"  
    Exit 1  
}
```

Codebeispiele

SCIM/Viya: Benutzer anlegen

```
$headers = @{{
    'Authorization' = "Bearer $access_token"
    'Content-Type' = 'application/scim+json'
}}

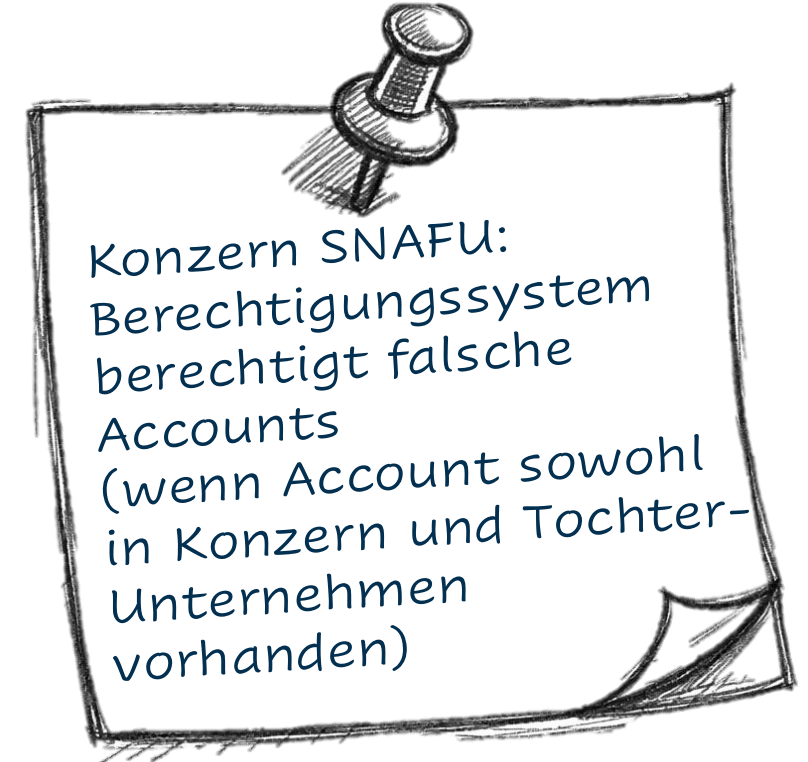
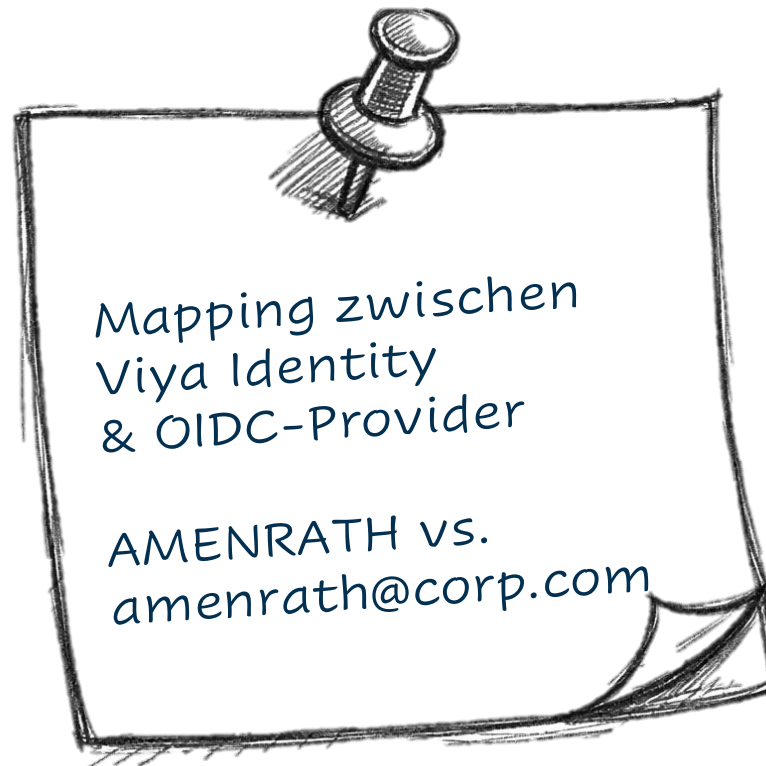
# create json body
$body = @{{
    schemas = @"urn:ietf:params:scim:schemas:core:2.0:User"
    externalId = $user.externalId
    userName = $user.userName
    displayName = $user.displayName
    emails = @(
        @{{
            primary = $true
            value = $user.email
            type = "work"
        }}
    )
}} | ConvertTo-Json
```

```
# send request
$create_user = Invoke-RestMethod `
    -Uri "$viya_url/identities/scim/v2/Users" `
    -Headers $headers `
    -Body $body `
    -Method Post `
    -SkipCertificateCheck `
    -SkipHttpErrorCheck `
    -StatusCodeVariable "statuscode"

if ($statuscode -in (200, 201, 204)) {
    Write-Output "✅ user $($user.externalId) created"
}
else {
    Write-Output "❌ user $($user.externalId) could not be created"
}
```

04 | Überraschung

Neue Anforderungen



05 | Fazit

Fazit

- ✓ SCIM-Schnittstelle lässt sich mit wenig Aufwand selbst bauen
- ✓ Vorgehen hat sich **bewährt**
- ✓ **Flexibel, leicht anpassbar und erweiterbar**
- ✓ **Erfolgreicher Einsatz** in weiteren Kundenprojekten



Fragen?



Andreas Menrath
Softwarearchitekt

✉ Andreas.Menrath@analytical-software.de

in <https://www.linkedin.com/in/andreasmenrath/>

